Solution: Longest Substring Without Repeating Characters (medium)

Problem Statement

Given a string, identify the length of its longest segment that contains distinct characters. In other words, find the maximum length of a substring that has no repeating characters.

Examples:

1. **Example 1**:
    - Input: "abcdaef"
    - Expected Output: 6
    - Justification: The longest segment with distinct characters is "bcdaef", which has a length of 6.
2. **Example 2**:
    - Input: "aaaaa"
    - Expected Output: 1
    - Justification: The entire string consists of the same character. Thus, the longest segment with unique characters is just "a", with a length of 1.
3. **Example 3**:
    - Input: "abrkaabcdefghijjxxx"
    - Expected Output: 10
    - Justification: The longest segment with distinct characters is "abcdefghij", which has a length of 10.

Algorithm Description:

To solve the problem, iterate through the characters of the given string while maintaining a HashSet to track the characters already encountered. As we traverse the string, each character is checked against the HashSet. If the character is not present in the HashSet, it indicates no repetition, and we add it to the HashSet and continue. However, if a character is already in the HashSet, it means a repetition has occurred. At this point, we update the length of the longest substring found so far (if necessary), and modify the HashSet to remove the characters up to and including the repeated character. This process continues until we have traversed the entire string. The final result is the length of the longest substring without repeating characters.

1. **Initialization**: Begin with two pointers, `start` and `end`, both at the start of the string. The hashset will initially be empty.

2. **Sliding Window Expansion**: Progressively move the `end` pointer to the right until you come across a character that's already in the hashset, indicating a repetition.

3. **Adjusting Start Pointer**: Upon detecting a repeated character, increment the `start` pointer by one position and remove the character at the `start` position from the hashset. This action ensures that the window only contains unique characters.

4. **Result Calculation**: At each step, calculate the length of the current window (from `start` to `end`). Keep track of the maximum length observed.

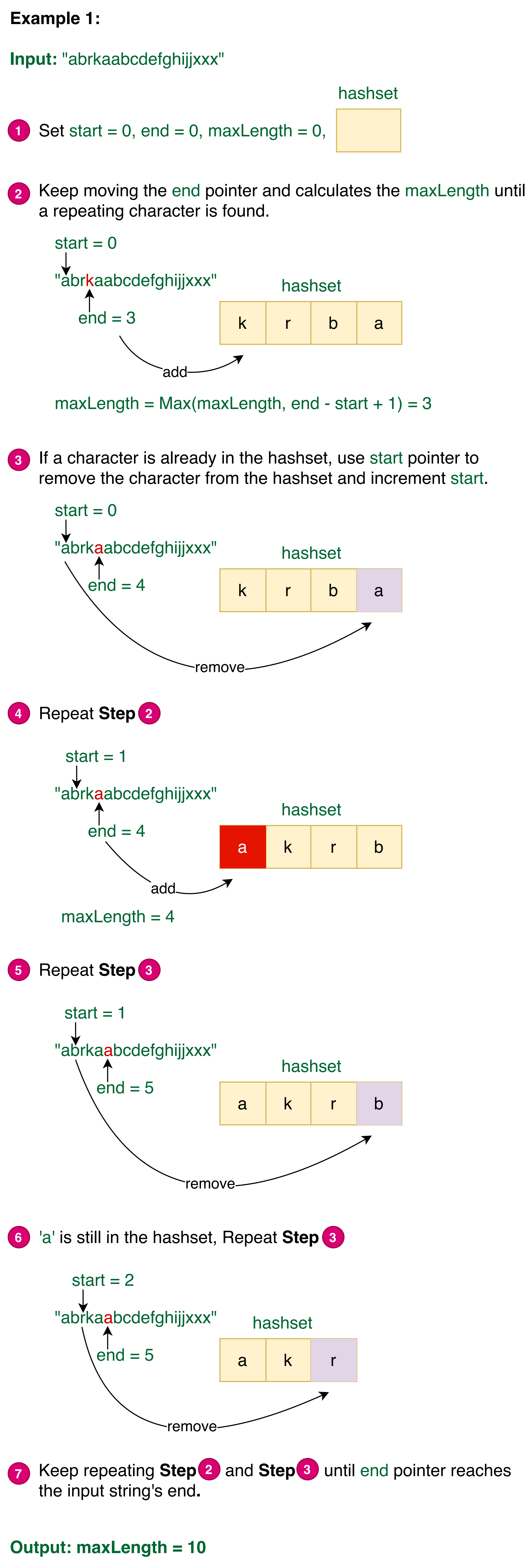
By the end of this process, the maximum length observed will be the length of the longest segment of unique characters in the string.

Algorithm Walkthrough:

Given the string "abrkaabcdefghijjxxx":

- Initialize `start` and `end` to 0, and an empty hashset.
- As you move `end` from 0 to the end of the string:
    - When `end` is at position 4 (character 'a'), since 'a' is already in the hashset, we will remove the character at position 0 ('a') from the hashset and move `start` to position 1.
    - Continue this process, always ensuring the characters between `start` and `end` are unique.
    - Calculate the length of the segment at each step (start -> end) and update the maximum length.
- The maximum length observed will be 10, corresponding to the segment "abcdefghij".

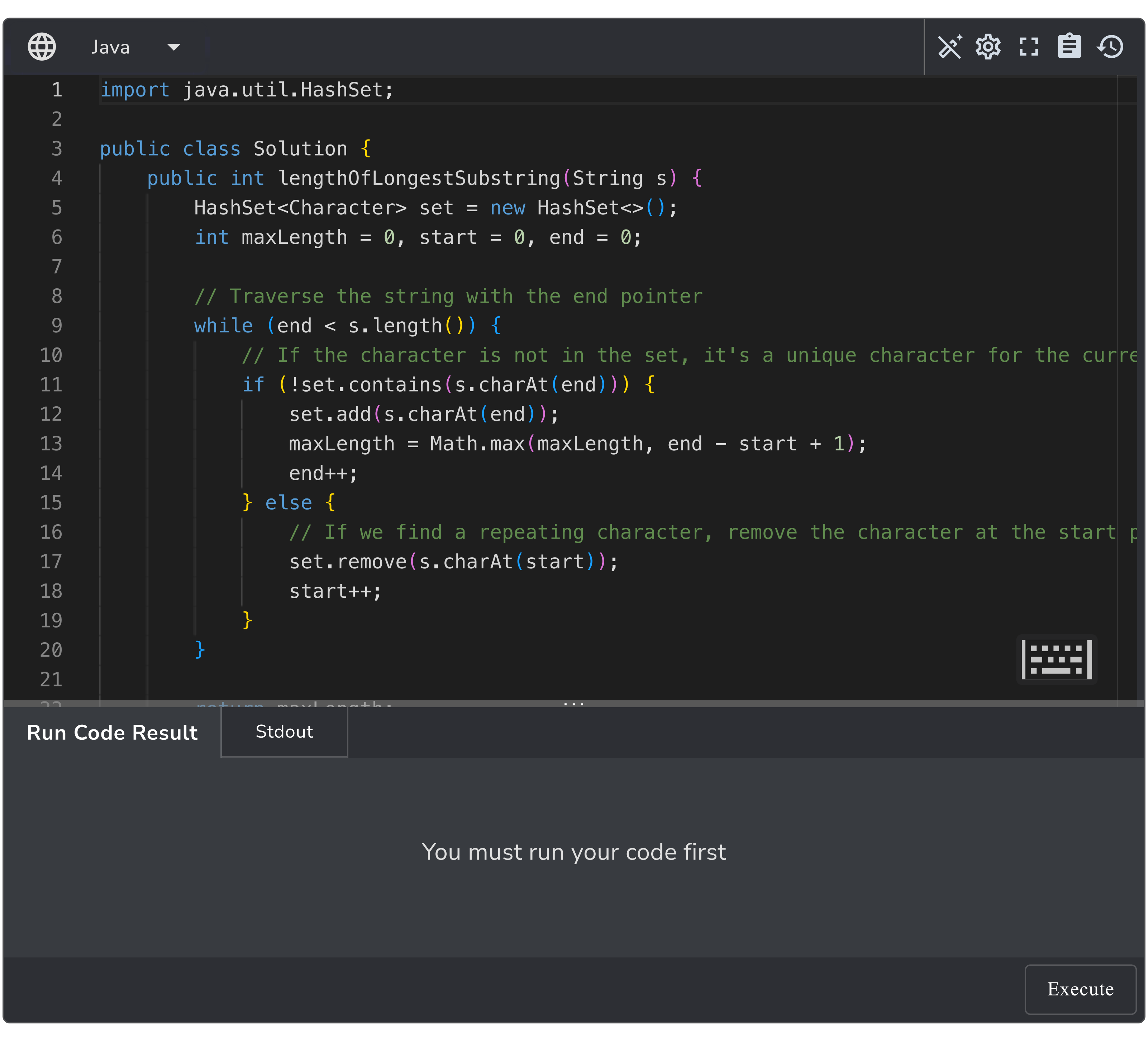
Here is the visual representation of the algorithm:


Example 1:
Input: "abrkaabcdefghijjxxx"
hashset
1 Set start = 0, end = 0, maxLength = 0,
2 Keep moving the end pointer and calculates the maxLength until a repeating character is found.
start = 0
"abrkaabcdefghijjxxx"
end = 3
hashset
k r b a
add
maxLength = Max(maxLength, end - start + 1) = 3
3 If a character is already in the hashset, use start pointer to remove the character from the hashset and increment start.
start = 0
"abrkaabcdefghijjxxx"
end = 4
hashset
k r b a
remove
4 Repeat Step 2
start = 1
"abrkaabcdefghijjxxx"
end = 4
hashset
a k r b
add
maxLength = 4
5 Repeat Step 3
start = 1
"abrkaabcdefghijjxxx"
end = 5
hashset
a k r b
remove
6 'a' is still in the hashset, Repeat Step 3
start = 2
"abrkaabcdefghijjxxx"
end = 5
hashset
a k r
remove
7 Keep repeating Step 2 and Step 3 until end pointer reaches the input string's end.
Output: maxLength = 10

Longest Substring Without Repeating Characters

Code

Here is the code for this algorithm:



```java
import java.util.HashSet;

public class Solution {
    public int lengthOfLongestSubstring(String s) {
        HashSet<Character> set = new HashSet<>();
        int maxLength = 0, start = 0, end = 0;

        // Traverse the string with the end pointer
        while (end < s.length()) {
            // If the character is not in the set, it's a unique character for the curre
            if (!set.contains(s.charAt(end))) {
                set.add(s.charAt(end));
                maxLength = Math.max(maxLength, end - start + 1);
                end++;
            } else {
                // If we find a repeating character, remove the character at the start p
                set.remove(s.charAt(start));
                start++;
            }
        }

```

You must run your code first

Time Complexity

1. **While Loop**: The algorithm uses a sliding window approach with two pointers, `start` and `end`. In the worst-case scenario, both pointers traverse the entire length of the string. Since each pointer can move from the beginning to the end of the string, the time complexity is O(2n), where n is the length of the string. However, in big O notation, constants are dropped, so the time complexity is O(n).

2. **HashSet Operations**: The operations of adding, deleting, and checking for the existence of an element in a HashSet (or Set in some languages) are O(1) on average. Therefore, these operations don't add any significant overhead to the time complexity.

Combining the above points, the overall time complexity of the algorithm is **O(n)**.

Space Complexity

1. **HashSet**: The space complexity is determined by the size of the HashSet. In the worst-case scenario, the HashSet will store all unique characters of the string. Since the set of possible characters is fixed (assuming the ASCII character set, which has 128 characters, or the extended ASCII set, which has 256 characters), the space complexity is O(min(n, m)), where n is the length of the string and m is the character set size (either 128 or 256). For most strings, n will be much larger than m, so the space complexity is effectively **O(m)**, which is a constant.

Therefore, the overall space complexity of the algorithm is **O(1)**, as it's constant and does not grow with the size of the input string.

← Previous
Problem 4: Longest Substring Without Repeating Characters (Medium)

Next →
Introduction To Heap

☐ Mark as Completed